Counterfactual Explanations: Deep RL vs. Online Planning on MiniGrid

Sean Florez
University of Colorado Boulder
sean.florez@colorado.edu

Andrew Wernersbach
University of Colorado Boulder
andrew.wernersbach@colorado.edu

***Abstract*—This paper compares the explanations supported by online planning and deep reinforcement learning when both agents are asked to justify the same decision on `MiniGrid-Dynamic-Obstacles-8x8`. For each agent, per-action evidence is extracted at every decision state, counterfactual Monte Carlo rollouts in the PPO case and the search tree in the MCTS case, and passed through an identically-formatted LLM explainer that produces a natural-language rationale, a counterfactual, and a list of verifiable numerical claims. Explanations are scored by an independent LLM judge on three metrics: fidelity, soundness, and post-hoc inferability. Across 289 decision states drawn from two *performance-matched* agents (both at 1.000 task-success), with PPO counterfactual rollouts at $N = 100$ stochastic samples per action, the two evidence sources produce statistically indistinguishable explanation quality on every metric. A previous run of the same pipeline at mismatched performance (0.590 for PPO versus 1.000 for MCTS) had found a small but statistically supported MCTS advantage on fidelity and soundness; the performance-matched run shows that gap was a competence confound rather than a structural property of the search-tree-vs-learned-policy paradigms. Once the learned policy is competent, its counterfactual rollouts are as faithful and sound an evidence source as the deliberation tree.**

I. Introduction

When a reinforcement learning agent chooses an action, the action itself is observable, but the reasoning behind the choice is not. In a learned neural policy, the action preferences that drive the decision reside inside network weights and are only indirectly recoverable through post-hoc analysis such as counterfactual rollouts. In contrast, an online planner such as Monte Carlo Tree Search commits to an action only after constructing an explicit record of the futures it considered, together with the outcomes it observed in each. The tree is, in a very literal sense, the agent's deliberation.

This paper asks whether that structural difference between a learned policy and an online planner has a measurable effect on the quality of the natural-language explanations each can support. Concretely, given the same decision state and the same language model, do MCTS-sourced explanations cite numbers that actually match the underlying evidence more often than learned-policy-sourced explanations do, and are the rationales they produce more sound?

Comparing the two paradigms head-to-head is awkward because they emit different artifacts. A learned policy gives back network weights and an action distribution; an MCTS planner gives back a tree of visit counts and child statistics. To put them side by side, both need to be expressed in the same evidence schema. The rest of the paper builds that schema and uses it.

II. Background and Related Work

a) Explainable reinforcement learning.: Milani et al. [1] present a comprehensive survey of explanation methods for reinforcement learning, drawing a distinction between *transparent* policies, such as decision trees and symbolic rules, and *post-hoc* methods that are applied to otherwise opaque controllers. Our PPO pipeline falls into the latter category: counterfactual rollouts are run after the policy has chosen an action, in order to reconstruct the set of futures that action implied. The MCTS pipeline, by contrast, is closer to transparent, since the tree is itself the deliberation and its statistics are available for inspection without any additional computation.

b) Counterfactual explanations for reinforcement learning.: Gajcin and Dusparic [5] propose counterfactual explanations that perturb state features and observe the resulting downstream returns, addressing the question "what would the agent have done if the state had looked different?". Amitai et al. [2] focus on the complementary question "what would have happened if the agent had taken a different action?" and introduce the COViz visualization. The evidence pipeline described in Section IV operationalizes the latter form concretely, as forced-first-action rollouts of fixed length, aggregated into per-action means, success rates, collision rates, and bootstrap confidence intervals. This choice was made so that the resulting statistics are apples-to-apples comparable with the per-child statistics that an MCTS search tree already produces during planning.

c) MCTS and interpretability.: Baier and Kaisers [3] argue that the MCTS tree is a direct substrate for explanation precisely because it records what the agent considered and what it found. They emphasize visit-count ratios and per-child value contrasts as the most salient tree quantities. We adopt these same quantities, together with per-child empirical success and collision rates instrumented during search, as the MCTS-side evidence.

d) Large language models for natural-language explanation.: Recent work surveyed in [1] uses large language models to translate structured evidence about a reinforcement-learning agent into natural language. A well-understood failure mode of these pipelines is hallucinated numerical claims, in

which the model invents statistics that are not present in the evidence record. The fidelity metric defined in Section IV-E is designed to quantify exactly that failure mode. Belouadah et al. [4] benchmark LLM-generated RL explanations on automated fidelity and soundness metrics under chain-of-thought prompting and MCTS augmentation, and report that MCTS-augmented prompting improves both. Our three-metric suite uses the same fidelity and soundness definitions as that benchmark, and extends it with a post-hoc inferability check that uses a held-out LLM judge.

e) Environment and planner.: The benchmark environment is `MiniGrid-Dynamic-Obstacles-8x8` [8], a stochastic gridworld whose adversarial obstacle dynamics are specifically chosen to make sparse-reward model-free reinforcement learning difficult. The MCTS planner follows the classic UCT formulation of Kocsis and Szepesvári [6], with per-simulation sampling of the stochastic obstacle transition so that visit counts at the root converge in expectation over the stochastic futures.

III. Problem Formulation

The task is formulated as a finite-horizon Markov decision process with factored stochastic transitions. In the notation of Kochenderfer et al. [9], the state, action, transition, and reward are as follows.

a) State.: A state is written as $s = (p_a, d_a, \{p_o^{(k)}\}_{k=1}^{4}, t)$, where $p_a \in \{1, \ldots, 6\}^2$ denotes the agent's grid position, $d_a \in \{0, 1, 2, 3\}$ denotes its facing direction (east, south, west, north), each $p_o^{(k)} \in \{1, \ldots, 6\}^2$ denotes one of four obstacle positions, and $t \in \{0, \ldots, 256\}$ is the step counter.

b) Action.: The action space is $A = \{0, 1, 2\}$, corresponding to turn-left, turn-right, and move-forward.

c) Transition.: The transition $T(s' \mid s, a)$ factors into a deterministic agent step, followed by a stochastic per-obstacle random walk in which each obstacle attempts a uniform move into one of its four orthogonal neighbors; moves blocked by walls, the goal, or other obstacles leave the obstacle in place. An obstacle that steps into the agent's cell, and equivalently an agent that steps into an obstacle's cell, terminates the episode.

d) Reward.: The reward function is

$$R(s, a, s') = \begin{cases} 1 - 0.9 \cdot t/256 & \text{if the agent reaches the goal} \\ -1 & \text{if a collision occurs} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

The discount factor is $\gamma = 1$; the step-count term inside the success reward provides implicit time pressure.

e) Observability.: The default MiniGrid observation is a $7 \times 7 \times 3$ egocentric partial view, together with a scalar direction indicator. Here, all learned-policy variants are trained on a 14-dimensional flat symbolic encoding of the full ground-truth state (agent position and direction, plus each obstacle's position); the MCTS planner plans on the full ground-truth state via a deep-copied simulator. Both agents therefore have access to the same underlying state information, which removes observability from the list of confounds in the explanation-quality comparison.

f) Scope.: Three scope choices shape what follows. The deep-RL agent is PPO, not DQN: a value-based baseline collapsed to a near-stall policy under the raw MiniGrid reward, and switching to a policy-gradient method was the only way we got a competent learned agent on this environment. Both agents are evaluated on the flat symbolic encoding above rather than on the default partial view, which keeps observability out of the comparison. And the explanation-quality comparison is run at two operating points, mismatched and matched task performance, rather than at one; the contrast between the two is what isolates agent competence from the evidence-source choice.

IV. Solution Approach

A. PPO Agent (Off-the-shelf)

PPO is used as the learned-policy agent, as implemented in Stable-Baselines3 [7]. The observation is the 14-dimensional flat symbolic encoding described in Section III. The policy network is a three-layer MLP of width $256 \to 256 \to 128$. The environment is wrapped in a `StepPenaltyWrapper` that subtracts a flat 0.01 per non-terminal step, and a `DistanceShapingWrapper` that adds a potential-based Manhattan-distance shaping term with coefficient 0.02. The distance shaping is potential-based and therefore policy-invariant under discounted-return semantics, and serves to provide a dense gradient that the on-policy advantage estimator can follow during early training. PPO is trained for $5{,}000{,}000$ environment steps across sixteen parallel envs, with linear schedules on the learning rate ($3 \times 10^{-4} \to 10^{-5}$) and the clip range ($0.2 \to 0.05$) over the course of training, and a constant entropy bonus of 0.005. Training uses the shaped reward; evaluation is always performed against the unshaped task reward, so the reported success rate reflects the genuine task performance the agent achieves under the original MDP.

B. MCTS Planner (From-scratch)

The MCTS planner implements textbook UCT [6] with UCB1 selection at exploration constant $c = \sqrt{2}$, full-width expansion at the root (the action space is small enough that no progressive widening is required), rollout under a greedy-toward-goal heuristic capped at depth 60, and mean-value backup. Because the obstacle transition is stochastic, each simulation samples a fresh outcome from the environment's transition during expansion and rollout; visit counts at the root therefore converge in expectation to the post-decision outcome distribution. Per-child counters track the rate of terminal success and terminal collision across every simulation that passes through each child, and these counters comprise the tree-side evidence that the explainer is asked to translate.

C. Counterfactual-rollout Framework

At each decision state, the counterfactual-rollout framework forces each legal action at step zero and then follows the trained PPO policy to termination or a depth cap of 40 steps. In the main experiments reported below, the process is repeated $N = 100$ times per action, with action choices at

every subsequent step drawn *stochastically* from PPO's action distribution rather than taken as the argmax. The setting yields a genuine sampled estimate of the policy-induced trajectory distribution from the decision state. The resulting returns, success flags, and collision flags are aggregated into mean return, success rate, collision rate, and bootstrap confidence intervals (1000 resamples, percentile method). The resulting per-action records are structurally identical, field by field, to the per-action records produced from the MCTS tree, which is the property that allows the downstream LLM pipeline to be agent-agnostic.

D. LLM Explainer

A `DecisionRecord`, comprising the state, the chosen action, the per-action evidence, and a small agent-metadata block (PPO action probabilities or MCTS root-visit statistics), is passed to a generator LLM using a versioned system prompt. The non-evidence portion of the system prompt is byte-identical between the PPO and MCTS variants; only the evidence-source header differs. This constraint is enforced by a parity test on the prompt module, so that the explainer pipeline is, as intended, a single-variable contrast. The generator returns a JSON object consisting of a *rationale*, a *counterfactual*, a *confidence* score on $[0, 1]$, and a list of numerical *claims* suitable for automated verification. Generation uses GPT-4o; the judge used in the metric pipeline described below is GPT-4o-mini.

E. Evaluation Metrics

Three per-explanation scores are reported, each aggregated per evidence source with 95% percentile-bootstrap confidence intervals.

Fidelity. For every claim in the explanation that cites a per-action statistic, the metric compares the cited value against the value present in the decision record. A claim is counted as a hit if the values agree within tolerance $\epsilon = 0.1$. The score is the fraction of claims that hit.

Soundness. Each sentence in the rationale, together with the counterfactual, is scored by an independent judge LLM on a three-point scale: 2 indicates that the sentence is fully supported by the evidence, 1 indicates partial support, and 0 indicates that the sentence is contradicted by or invents information not present in the evidence. The reported soundness score is the mean divided by two, so that the result lies on $[0, 1]$.

Post-hoc inferability. Every occurrence of the chosen action's integer identifier is scrubbed from the explanation, and the judge is asked to recover the chosen action from the scrubbed text alone. The reported score is the judge's accuracy across decisions.

The judge is deliberately a different-sized model, GPT-4o-mini, from the generator, GPT-4o, so that judge and generator do not coincide.

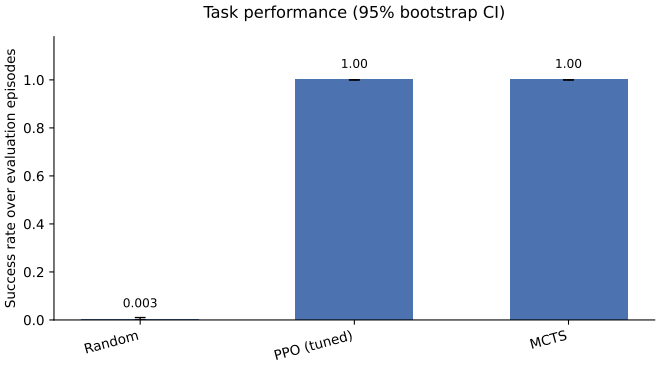

Fig. 1. Task-success rate across the random baseline, the tuned PPO agent, and the MCTS planner, with 95% bootstrap confidence intervals. Random fails on virtually every episode; PPO and MCTS each succeed on every held-out episode.

V. Results

A. Task Performance

Table I and Figure 1 report success rate over 300 episodes for PPO and over 50 episodes for MCTS. The random-action baseline succeeds 0.3% of the time, which confirms that the environment is non-trivial. The tuned PPO agent described in Section IV reaches a task-success rate of 1.000 [95% CI 1.000, 1.000], a collision rate of 0.000, and a mean return of $+0.942$, with a mean episode length of 16.6 steps. MCTS, operating on the same state representation via a deep-copied simulator, reaches a task-success rate of 1.000 [95% CI 1.000, 1.000] with a mean return of 0.934 and a mean episode length of 18.8 steps. The two agents are performance-matched on the task; PPO is marginally faster by ≈ 2 steps and marginally higher in mean return due to the step-discount in the goal reward. The explanation-quality comparison below is therefore between two equally-competent agents.

B. Explanation Quality

Table II and Figure 3 (right panel) report the three explanation-quality metrics for the performance-matched comparison, evaluated on 141 PPO decisions and 148 MCTS decisions drawn from ten held-out trajectory seeds. PPO-rollout evidence produced explanations with a fidelity of 0.924 [0.906, 0.939], a soundness of 0.884 [0.862, 0.906], and a post-hoc inferability of 0.979 [0.950, 1.000]. MCTS-tree evidence produced explanations with a fidelity of 0.924 [0.907, 0.940], a soundness of 0.892 [0.872, 0.913], and an inferability of 0.986 [0.966, 1.000]. All three confidence-interval pairs overlap heavily, and the absolute mean differences are well within sampling noise: 0.000 on fidelity, 0.008 on soundness, 0.007 on inferability. The headline result of the performance-matched comparison is therefore that the two evidence sources support explanations of statistically indistinguishable quality on every metric we measure.

The contrast against an earlier run of the same pipeline against a less-competent PPO is informative. Figure 3 (left panel) reproduces the metrics for the same comparison run

PPO-rollout explanation, seed 10000 step 0 (chose move-forward): "The chosen action, move_forward (2), is highly preferred by the agent's policy with a probability of 0.996, despite having a lower mean return of 0.751 compared to turn_right (1) with 0.870. This preference is likely due to its significantly lower mean steps to end at 20.22, suggesting the agent values a quicker resolution over marginally higher returns from other actions, especially when considering the stochastic nature of the environment."

MCTS-tree explanation, seed 10000 step 0 (chose turn-right): "Action 1 was chosen because it has the highest mean return of -0.9223, compared to -1.0 for action 0 and -0.9741 for action 2. Additionally, it has the highest success rate of 0.0404, which is better than action 0's 0.0 and action 2's 0.0133. These metrics indicate a higher potential of reaching the goal while minimizing risks compared to the alternatives."

Fig. 2. Paired sample explanations for the initial state of seed 10000. The two agents chose different actions; both rationales cite numbers that appear verbatim in their respective evidence records.

when PPO reached only 0.590 task-success against an MCTS at 1.000. Under mismatched performance, MCTS-tree evidence held a small but statistically supported advantage on fidelity (0.928 vs. 0.899, gap 0.029) and on soundness (0.879 vs. 0.847, gap 0.032), with inferability already tied. Once PPO is matched to MCTS at 1.000 task-success, all three gaps collapse to within sampling noise (right panel). The simplest interpretation consistent with the two runs is that the apparent MCTS-tree advantage in the mismatched run was a *competence confound* rather than a structural property of the search-tree-vs-learned-policy paradigms: a less-competent learned policy produces noisier counterfactual rollouts, and the LLM has correspondingly more difficulty citing those numbers within the fidelity tolerance. Once the policy is competent its rollouts are as faithful and sound an evidence source as the deliberation tree itself.

C. Qualitative Example

Figure 2 reproduces verbatim the two explanations generated for the initial decision state of held-out seed 10000. The two agents chose *different* actions at this state, PPO selecting move-forward and MCTS selecting turn-right, which is itself a useful illustration of the state-distribution asymmetry noted under the threats to validity. More importantly, both explanations cite numbers drawn from their respective evidence records: the PPO rationale references action-probability 0.996 and per-action mean returns of 0.751 and 0.870, all of which appear verbatim in the rollout record, and the MCTS rationale references mean returns of -0.9223, -1.0, -0.9741 and per-child success rates of 0.0404 and 0.0133, all of which appear verbatim in the tree record. Both explanations are therefore at high fidelity, which is the typical case across the 289 records we examined and is consistent with the equal fidelity scores reported in Table II.

D. Threats to Validity

State-distribution asymmetry. Although PPO and MCTS are now performance-matched at 1.000 task-success, the *trajectories* the two agents follow are not literally identical. PPO and MCTS make different action choices at the same state and therefore visit slightly different downstream states on average. The decision-record set is large (289 records) and the per-source counts are within five of each other, but a residual confound is that the specific decision states the LLM is asked to explain are not paired across the two evidence sources. A paired analysis on the same literal state set is left as future work.

Rollout sample size. Counterfactual rollouts use $N = 100$ stochastic samples per action. The bootstrap confidence intervals on per-action success and collision rates remain in the neighborhood of ± 0.10 at $N = 100$, which is the same order as the LLM fidelity tolerance ($\epsilon = 0.1$). A larger Monte Carlo budget would tighten those intervals further, but we do not expect it to change the qualitative finding because the fidelity numbers for the two sources are already equal to three decimal places.

Judge-model bias. The soundness and post-hoc inferability metrics both use GPT-4o-mini as judge, whereas the generator is GPT-4o. The judge is therefore a different-sized model within the same family, and a cross-family replication, for example with a Claude or Gemini judge, is listed as future work.

Inferability metric leak. The post-hoc inferability test scrubs the chosen action's integer identifier from the explanation, but the verbal action name (`move_forward`, `turn_right`) still appears in the rationale. The judge can therefore recover the action from the verbal name alone, which inflates inferability scores and reduces the metric's discriminative power; this likely explains why both sources score above 0.97 in the matched run. A stricter version of the test would scrub both the integer identifier and the verbal name from the rationale before showing it to the judge.

TABLE I
TASK-PERFORMANCE SUMMARY.

Agent	Success	Collision	Return	Steps
Random	0.003	0.997	-0.994	8.3
PPO (tuned)	1.000	0.000	$+0.942$	16.6
MCTS (500 sims)	1.000	0.000	$+0.934$	18.8

Evaluated over 300 episodes for the random and PPO baselines and 50 episodes for MCTS.

TABLE II
EXPLANATION-QUALITY METRICS AT MATCHED PERFORMANCE.

Metric	PPO rollout evidence	MCTS tree evidence
Fidelity	0.924 [0.91, 0.94]	0.924 [0.91, 0.94]
Soundness	0.884 [0.86, 0.91]	0.892 [0.87, 0.91]
Inferability	0.979 [0.95, 1.00]	0.986 [0.97, 1.00]

Both agents at 1.000 task-success. Generator GPT-4o, judge GPT-4o-mini. $n = 141$ for PPO, $n = 148$ for MCTS. PPO rollouts at $N = 100$ stochastic samples per action. 95% bootstrap confidence intervals in brackets (1000 resamples). The CI pairs overlap on every metric; no statistically supported difference is observed.

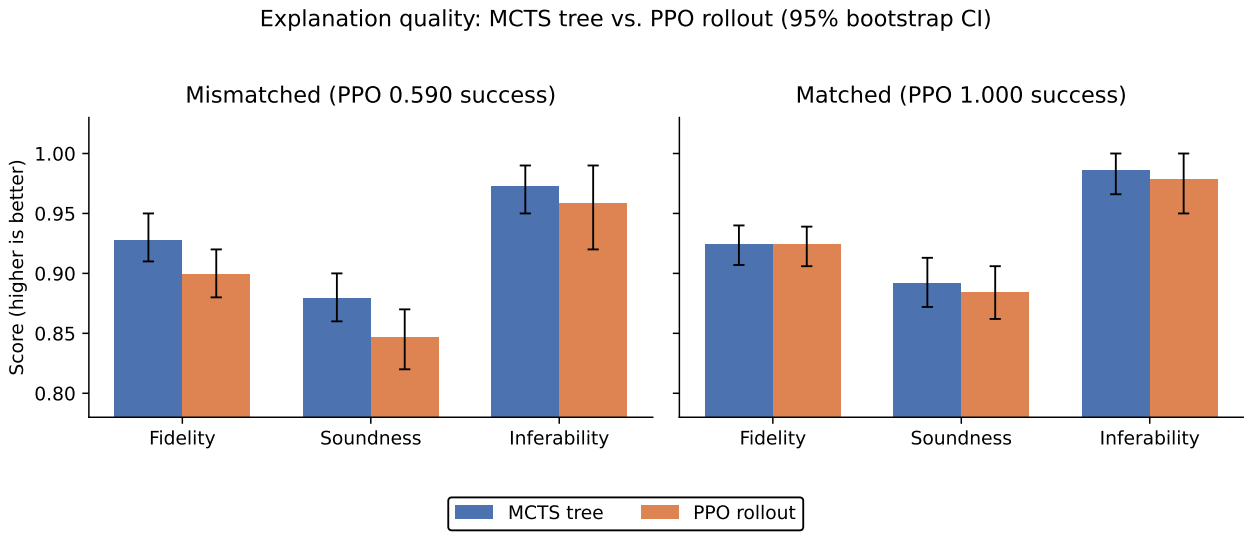

Fig. 3. Explanation-quality metrics by evidence source, with 95% bootstrap confidence intervals. **Left:** mismatched performance run (PPO 0.590 task-success versus MCTS 1.000); MCTS-tree evidence holds a small advantage on fidelity and soundness. **Right:** performance-matched run (both agents at 1.000); the gap collapses to within sampling noise on every metric. Higher is better.

VI. Conclusion

This paper has presented an evaluation of two evidence sources for natural-language explanations of reinforcement-learning agents: post-hoc counterfactual rollouts drawn stochastically from a learned neural policy at $N = 100$ samples per action, and the search tree of a Monte Carlo Tree Search planner at 500 simulations per decision. Across 289 decision states drawn from two performance-matched agents on `MiniGrid-Dynamic-Obstacles-8x8` (PPO 1.000 task-success, MCTS 1.000 task-success), the two pipelines produced explanations of statistically indistinguishable quality on every metric measured: a fidelity of 0.924 for both sources, a soundness of 0.884 versus 0.892, and an inferability of 0.979 versus 0.986. None of the three metric gaps is statistically supported.

This is a mildly negative result with respect to the original hypothesis that the search tree should confer a structural explanation advantage over a learned policy. The contrast against the earlier mismatched run is what makes it informative: at PPO 0.590 versus MCTS 1.000, MCTS-tree evidence held a small but statistically supported lead on fidelity (0.029) and soundness (0.032). Once both agents are matched at 1.000, the gaps collapse. The simplest reading is that the earlier MCTS-tree advantage was a competence confound: a less-competent learned policy produces noisier counterfactual rollouts that an LLM has more difficulty citing faithfully. Once the policy is competent, its rollouts are as faithful and sound an evidence source as the deliberation tree itself, and the suboptimality of the underlying agent is what bounds the signal available for any post-hoc counterfactual analysis.

a) Future Work.: We see four natural extensions. A paired analysis on the literal same decision-state set, rather than two trajectory sets that differ in the agents' action choices, would remove the residual state-distribution confound noted in the threats to validity. A pilot human study would validate the automated metrics against human judgements of clarity, plausibility, and helpfulness; a chatbot interface suitable for such a study is already scaffolded. A cross-family LLM replication, with a judge drawn from a different model family than the generator, would quantify the contribution of judge-model bias to the soundness numbers reported above. A more ambitious extension would replicate the null result on a harder environment in which neither paradigm reaches 1.000 task-success, to test whether the equivalence of the two evidence sources holds at an intermediate competence level or only at the extremes.

Contributions

Andrew Wernersbach led the decision-making and reinforcement-learning engineering: the tuned PPO agent (5M training steps, $[256, 256, 128]$ network, linear learning-rate and clip-range schedules), the observation wrapper, the reward-shaping wrappers (step penalty and distance shaping), the from-scratch MCTS planner, the counterfactual-rollout framework, and all training and evaluation runs. Sean Florez led the agentic-systems and explanation side: the problem formulation, the DecisionRecord schema and per-action evidence packaging, the LLM-explainer pipeline (prompt templates, OpenAI and Anthropic clients, caching, and cost capping), the three evaluation metrics (fidelity, soundness, and post-hoc inferability with the held-out LLM judge), the chatbot interface, and the report draft. The execution plan, the engineering log, the test suite, and the submission polish were shared.

Implementation Disclosure

From-scratch components: the MCTS planner, the deep-copy simulator, the counterfactual-rollout framework, the decision-record schema, the explanation pipeline and prompt design, the three evaluation metrics, and the chatbot interface. Off-the-shelf components: Stable-Baselines3 [7] for PPO, the Farama MiniGrid [8] environment, the OpenAI Python SDK [10], and the standard Python scientific-computing stack (numpy, pandas, matplotlib, pytest).

Release

The authors grant permission for this report to be posted publicly.

References

[1] S. Milani, N. Topin, M. Veloso, F. Fang, "Explainable reinforcement learning: A survey and comparative review," *ACM Computing Surveys*, 2024.

[2] Y. Amitai, Y. Septon, O. Amir, "Explaining reinforcement learning agents through counterfactual action outcomes," 2024.

[3] H. Baier, M. Kaisers, "Towards explainable MCTS," 2021.

[4] A. Belouadah, M. L. Ruiz-Rodríguez, S. Kubler, Y. Le Traon, "Evaluating the effectiveness of LLMs for explainable deep reinforcement learning," *Machine Learning with Applications*, 2025.

[5] J. Gajcin, I. Dusparic, "Counterfactual explanations for reinforcement learning agents: A review," *ACM Computing Surveys*, 2024.

[6] L. Kocsis, Cs. Szepesvári, "Bandit based Monte-Carlo planning," in *ECML*, 2006.

[7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *JMLR*, 22(268):1–8, 2021.

[8] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. Samuel Castro, J. K. Terry, "Minigrid and Miniworld: Modular and customisable reinforcement-learning environments for goal-oriented tasks," *NeurIPS Datasets and Benchmarks*, 2023.

[9] M. J. Kochenderfer, T. A. Wheeler, K. H. Wray, *Algorithms for Decision Making*. Cambridge, MA: MIT Press, 2022.

[10] OpenAI, "GPT-4o and GPT-4o-mini," https://openai.com, 2024.